

Détection d'intrusion avec KNN Master Réseaux et Systèmes – Cours de Machine Learning

2025-06-24

Objectif pédagogique

Comprendre l'algorithme des K plus proches voisins (KNN) et l'appliquer dans un contexte concret : la détection d'attaques réseau à partir de données de trafic.

Contexte

Les systèmes en réseau sont exposés à différents types d'attaques : scan de ports, déni de service (DoS), injections, etc. Des systèmes de détection d'intrusion (IDS) sont utilisés pour détecter ces comportements anormaux.

Dans ce TP, vous allez utiliser l'algorithme KNN pour classer des connexions réseau comme normales ou malveillantes à partir d'un jeu de données représentatif.

Jeu de données

Le jeu de données utilisé est **NSL-KDD**, un jeu de données de référence pour la détection d'intrusion.

Téléchargement (fichiers CSV déjà préparés) :

https://github.com/defcom17/NSL_KDD

Fichiers utiles : - **KDDTrain+.csv** : ensemble d'entraînement

- **KDDTest+.csv** : ensemble de test

Partie 1 : Préparation des données

1. Charger les données avec **pandas**.
 2. Afficher les premières lignes et vérifier les types de variables.
 3. Encoder les variables catégorielles (**protocol_type**, **service**, **flag**) avec **LabelEncoder** ou **pd.get_dummies**.
 4. Identifier la variable cible (**label**) : **normal** ou type d'attaque.
 5. Ajouter une nouvelle variable binaire **is_attack** où :
 - **normal** devient 0
 - toute autre valeur devient 1
 6. Séparer les données en **X** (caractéristiques) et **y** (**is_attack**).
 7. Diviser en ensemble d'entraînement et ensemble de test (80/20).
-

Partie 2 : Implémentation de KNN

1. Utiliser `KNeighborsClassifier` de `scikit-learn`.
 2. Entraîner le modèle avec les données d'entraînement.
 3. Prédire les classes sur les données de test.
 4. Évaluer la performance du modèle :
 - Précision
 - Rappel
 - F1-score
 - Matrice de confusion
-

Partie 3 : Expérimentation

1. Faire varier la valeur de `k` (1, 3, 5, 7, 11).
 2. Comparer les performances obtenues pour chaque valeur de `k`.
 3. Identifier la valeur optimale.
-

Partie 4 : Analyse

1. Avantages de KNN pour la détection d'intrusion.
 2. Inconvénients de KNN pour une utilisation en temps réel.
 3. Suggestions d'améliorations (par exemple : normalisation, réduction de dimension).
 4. Proposer d'autres algorithmes plus adaptés (ex. : SVM, Random Forest, Réseaux de neurones).
-

Livrable attendu

- Un fichier Jupyter Notebook clair et commenté, contenant :
 - Le code de traitement
 - Les résultats des expérimentations
 - Les réponses aux questions d'analyse
- Nom du fichier : `TP_KNN_IDS_NomPrenom.ipynb`